



**UNIVERSIDADE ESTADUAL DO CEARÁ
CENTRO DE CIÊNCIAS E TECNOLOGIA
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

GERALDO DUARTE DE MEDEIROS NETO - 1883885

JOÃO BATISTA ALVES DE SOUSA JÚNIOR - 1883853

RELATÓRIO DO TRABALHO DE PROGRAMAÇÃO E ALGORITMOS

FORTALEZA

2026.1

SUMÁRIO

1 INTRODUÇÃO.....	3
2 ESTRUTURA DO PROJETO.....	4
3 FUNÇÕES.....	7
3.1 Funções Auxiliares (auxiliares.c).....	7
3.2 Funções de Livros (livros.c).....	7
3.3 Funções de Usuários (usuarios.c).....	8
3.4 Funções de Empréstimos (emprestimos.c).....	8
3.5 Funções de Relatórios (relatorios.c).....	8
4 IDEIAS, PROBLEMAS E DECISÕES.....	10
4.1 Armazenamento Binário.....	10
4.2 Geração de IDs Únicos.....	10
4.3 Utilização do Makefile para compilação do projeto.....	10
4.3 Separação de trabalhos.....	10
5 CONCLUSÃO.....	11
REFERÊNCIAS.....	12

1 INTRODUÇÃO

O programa apresentado neste relatório é um sistema de gerenciamento de biblioteca desenvolvido na linguagem C. O sistema foi construído com base em quatro funcionalidades principais: gerenciamento de livros, gerenciamento de usuários, controle de empréstimos e devoluções, e geração de relatórios em formato de texto para visualização dos dados.

Além dessas funcionalidades, o programa dispõe de um menu principal interativo que direciona o usuário para cada módulo e suas respectivas subfunções, garantindo a persistência de dados por meio de arquivos binários e a estruturação das entidades do sistema por meio de structs. Todas as funções utilizadas foram desenvolvidas pela equipe ou importadas de bibliotecas padrão da linguagem C, conforme os conteúdos abordados na disciplina de Programação e Algoritmos.

Este relatório tem como objetivo apresentar toda a arquitetura do sistema e as decisões tomadas pela equipe ao longo do processo de desenvolvimento do projeto, cobrindo desde a estrutura de diretórios até os detalhes de implementação de cada módulo.

2 ESTRUTURA DO PROJETO

A estrutura do projeto é organizada em cinco diretórios principais, cada um responsável por um conjunto específico de artefatos do sistema. Essa divisão visa garantir clareza, modularidade e facilidade de manutenção ao longo do desenvolvimento.

Os diretórios são: assets (utilitários visuais — artes ASCII — para cada função do sistema), data (arquivos binários .dat que armazenam os registros persistentes), include (arquivos de cabeçalho .h com declarações de structs e funções), relatorios (pasta onde os relatórios .txt são salvos) e src (código-fonte .c com todas as implementações).

Os três arquivos de dados principais são: ListaLivros.dat (armazena registros de livros), ListaUsuarios.dat (armazena registros de usuários) e ListaEmprestimos.dat (armazena registros de empréstimos). A seguir, a Figura 1 ilustra a árvore completa de arquivos do projeto:



Figura 1 — Árvore de arquivos do projeto

struct Usuario	
matricula	: int
nome	: char[50]
curso	: char[50]
quant_emprestimos_ativos	: int

Figura 2 — Estrutura de Usuários

struct Livro	
id	: int
nome	: char[50]
autor	: char[50]
genero	: char[50]
ano	: int
quant_total	: int
quant_emprestado	: int
quant_disp	: int
total_emprestimos	: int

Figura 3 - Estrutura de Livros

struct Emprestimo	
id_emprestimo	: int
id_livro	: int
matricula_usuario	: int
data_retirada	: char[11]
data_prevista	: char[11]
data_devolucao	: char[11]
devolvido	: int

Figura 4 - Estrutura de Empréstimos

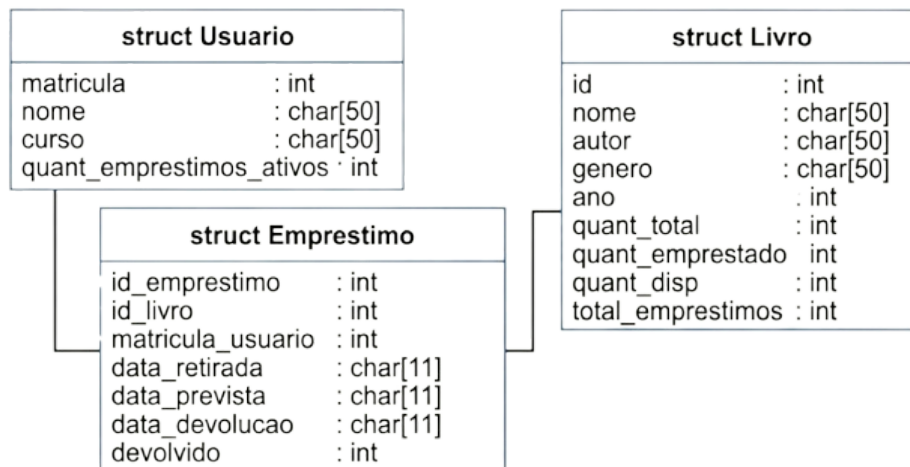


Figura 5 — Relação entre as estruturas

3 FUNÇÕES

3.1 Funções Auxiliares (auxiliares.c)

O módulo `auxiliares.c` concentra funções utilitárias reutilizadas em todo o sistema. As principais são:

- `pausar()`: aguarda o usuário pressionar ENTER para continuar a execução.
- `limparBuffer()`: limpa o buffer do teclado após leituras realizadas com `scanf`.
- `limparTela()`: limpa o terminal (`cls` no Windows, `clear` no Linux).
- `exibirTitulo()`: lê e exibe uma arte ASCII a partir de um arquivo `.txt`.
- `erroEntrada()`: exibe uma mensagem de erro padronizada ao usuário.
- `confirmar()`: solicita confirmação S/N do usuário antes de operações críticas.
- `obterDataAtual()`: obtém a data atual do sistema no formato `dd/mm/aaaa`.
- `obterDataFutura()`: calcula uma data futura a partir de um número de dias fornecido.
- `converterDataParaDias()`: converte uma data em string para número de dias, permitindo comparações.

3.2 Funções de Livros (livros.c)

O módulo `livros.c` gerencia o acervo bibliográfico. Suas funções são:

- `c_livros()`: cadastra novos livros com verificação de ID duplicado via alocação dinâmica (`malloc`).
- `p_livros()`: pesquisa livros por nome (busca parcial com `strstr`) ou por ID.
- `l_livros()`: lista todos os livros cadastrados no acervo.
- `a_livros()`: atualiza as informações de um livro existente a partir do seu ID.

- `r_livros()`: remove um livro do sistema, bloqueando a operação caso haja exemplares em empréstimo.
- `e_livros()`: exibe todos os empréstimos ativos associados a um livro específico.

3.3 Funções de Usuários (usuarios.c)

O módulo `usuarios.c` gerencia os usuários cadastrados no sistema:

- `c_usuarios()`: cadastra novos usuários com matrícula de 5 dígitos, garantindo unicidade.
- `p_usuarios()`: pesquisa usuários por nome ou número de matrícula.
- `l_usuarios()`: lista todos os usuários cadastrados.
- `a_usuarios()`: atualiza o nome ou o curso de um usuário existente.
- `r_usuarios()`: remove um usuário pelo número de matrícula.

3.4 Funções de Empréstimos (emprestimos.c)

O módulo `emprestimos.c` controla o ciclo de empréstimo e devolução de livros:

- `c_emprestimos()`: registra um novo empréstimo, verificando a disponibilidade do livro e o limite do usuário (máximo de 3 empréstimos ativos simultâneos).
- `p_emprestimos()`: pesquisa empréstimos por usuário ou por livro.
- `l_emprestimos()`: lista todos os empréstimos registrados no sistema, exibindo o status de cada um.
- `d_emprestimos()`: Registra a devolução de um empréstimos

3.5 Funções de Relatórios (relatorios.c)

O módulo `relatorios.c` gera relatórios analíticos em arquivos de texto:

- `lm_relatorio()`: gera relatório dos livros mais emprestados em ordem decrescente, utilizando o algoritmo bubble sort.

- `a_relatorio()`: gera relatório de empréstimos em atraso, comparando a data prevista de devolução com a data atual.
- `ad_relatorio()`: gera relatório do acervo disponível, listando livros com exemplares disponíveis para empréstimo.
- `h_relatorio()`: gera o histórico completo de empréstimos de um usuário específico

4 IDEIAS, PROBLEMAS E DECISÕES

4.1 Armazenamento Binário

A equipe optou por armazenar os dados em arquivos binários (.dat) com fwrite/fread em vez de arquivos de texto com fprintf/fscanf. Essa decisão foi motivada pela eficiência na leitura e escrita de structs completas: com arquivos binários, é possível ler um registro inteiro com uma única chamada a fread, eliminando a necessidade de parsing de texto e reduzindo significativamente o risco de erros de formatação.

4.2 Geração de IDs Únicos

Para garantir a unicidade dos identificadores, foi implementado um loop do-while que gera um ID aleatório, verifica se já existe entre os registros cadastrados na sessão atual e também no arquivo binário persistente. Em caso de colisão, um novo ID é gerado automaticamente. A função srand(time(NULL)) é chamada uma única vez antes do loop para evitar a geração de sequências repetidas entre sessões distintas.

4.3 Utilização do Makefile para compilação do projeto

Para garantir a eficiência no desenvolvimento do sistema, foi optado por utilizar o arquivo makefile para que o usuário tenha facilidade na compilação, execução, depuração e limpeza dos arquivos gerados. Além disso foi utilizado estrutura modular na compilação do projeto para auxiliar na construção do sistema a longo prazo.

4.3 Separação de trabalhos

Decidimos dividir o projeto entre os dois integrantes para que cada um pudesse aplicar os conceitos aprendidos. Geraldo Duarte ficou responsável principalmente pelas implementações relacionadas a usuários, empréstimos, e

funções auxiliares e João Batisa ficou responsável principalmente pelas implementações relacionadas a livros e relatórios e persistência de dados.

5 CONCLUSÃO

O sistema de gerenciamento de biblioteca desenvolvido em linguagem C atende plenamente aos requisitos propostos, implementando as quatro funcionalidades principais: gerenciamento de livros, de usuários, de empréstimos e geração de relatórios. O projeto consolidou o aprendizado de conceitos fundamentais como estruturas de dados (structs), alocação dinâmica de memória (malloc/realloc), manipulação de arquivos binários (fread/fwrite) e organização modular de código em C.

As principais dificuldades enfrentadas foram a manipulação correta de arquivos binários — especialmente para a atualização de registros no meio do arquivo — e a garantia de unicidade dos identificadores gerados aleatoriamente. Ambos os desafios foram superados com as estratégias descritas ao longo deste relatório.

O projeto demonstrou na prática conceitos fundamentais de programação em C e serviu como exercício completo de desenvolvimento de um sistema com persistência de dados, validação de entradas e interface de usuário via terminal, contribuindo de forma significativa para a formação técnica da equipe.

REFERÊNCIAS

FERNANDA, A.; DE, V. **Fundamentos da programação de computadores : algoritmos, Pascal, C/C++ e Java.** São Paulo: Pearson Prentice Hall, 2008.

DEITEL, Harvey M.; DEITEL, Paul J. C: Como Programar. 6. ed. Pearson, 2011. cppreference.com. C reference. Disponível em: <<https://en.cppreference.com/w/c>>. Acesso em: jun. 2026.

FRANK, D. Header Files In C. Disponível em: <https://medium.com/@Dev_Frank/libraries-in-c-b989c5d66ba2>.

GNU. GCC, the GNU Compiler Collection. Disponível em: <<https://gcc.gnu.org/>>. Acesso em: jun. 2026.

Text to ASCII Art Generator (TAAG). Disponível em: <<https://patorjk.com/software/taag/>>. Acesso em: jun. 2026.